

LAB 5 - Mandelbroot

Table A.1: Analysis

Cas 1 → Geometric decomposition: Block geometric data decomposition by columns

Code:

```
C/C++
void mandel_simple(int M[ROWS][COLS], double CminR, double CminI, double
CmaxR, double CmaxI, double scale_real, double scale_imag, int maxiter)
{
    #pragma omp parallel
    {
        int my_id = omp_get_thread_num();
        int num_threads = omp_get_num_threads();

        int x_start = my_id * (COLS / num_threads);
        int x_end = (my_id + 1) * (COLS / num_threads);

        int rem = COLS % num_threads;
        if (rem != 0) {
            if (my_id < rem) {
                x_start += my_id;
                x_end += (my_id + 1);
            } else {
                x_start += rem;
                x_end += rem;
            }
        }

        // Calculer
        for (int y = 0; y < ROWS; y++) {
            for (int x = x_start; x < x_end; x++) {
                M[y][x] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR,
CmaxI, x, y, scale_real, scale_imag, maxiter);
                if (output2histogram)
                    #pragma omp atomic
                    histogram[M[y][x] - 1]++;
                if (output2display) {
                    /* Scale color and display point */
                }
            }
        }
    }
}
```

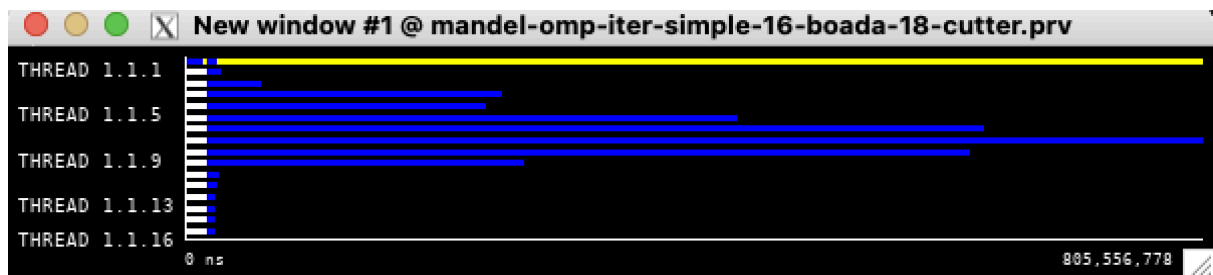

Overview of the Efficiency metrics in parallel fraction, $\phi=99.60\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	69.02%	39.47%	28.71%	26.73%	25.50%	24.85%	24.97%	24.95%	25.29%
Parallelization strategy efficiency	100.00%	69.06%	39.50%	28.77%	26.84%	25.64%	25.18%	25.45%	25.71%	26.01%
Load balancing	100.00%	69.07%	39.51%	28.79%	26.87%	25.67%	25.22%	25.50%	25.79%	26.12%
In execution efficiency	100.00%	99.99%	99.98%	99.95%	99.90%	99.88%	99.83%	99.78%	99.69%	99.59%
Scalability for computation tasks	100.00%	99.94%	99.92%	99.79%	99.61%	99.45%	98.69%	98.14%	97.07%	97.22%
IPC scalability	100.00%	100.00%	100.00%	100.00%	99.82%	99.76%	99.21%	98.82%	98.05%	98.51%
Instruction scalability	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.99%	99.98%	99.98%	99.98%
Frequency scalability	100.00%	99.94%	99.92%	99.80%	99.79%	99.70%	99.49%	99.33%	99.02%	98.71%

Table 2: Analysis done on Wed Dec 17 09:28:58 AM CET 2025, paco1114

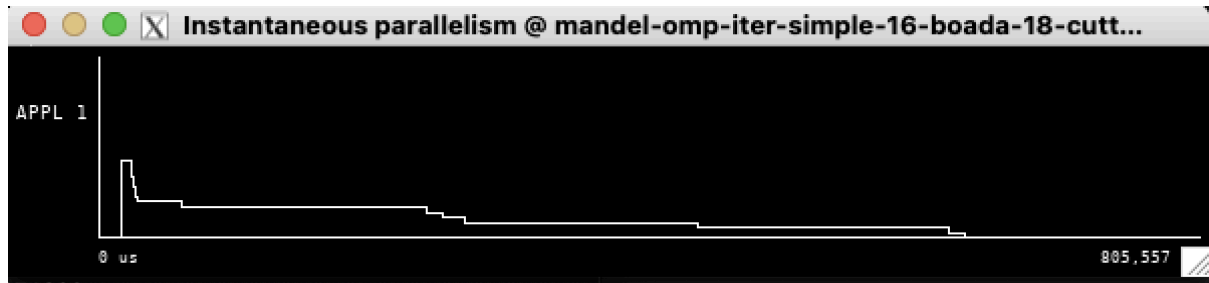
Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3229689.87	1615860.82	808041.75	404568.68	270199.87	202967.08	163635.84	137127.17	118828.36	103816.29
Load balancing for implicit tasks	1.0	0.69	0.4	0.29	0.27	0.26	0.25	0.26	0.26	0.26
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	114.5	156.47	1750139.91	1386301.95	993812.62	781760.27	642377.46	532146.06	456095.83	393629.06

Table 3: Analysis done on Wed Dec 17 09:28:58 AM CET 2025, paco1114

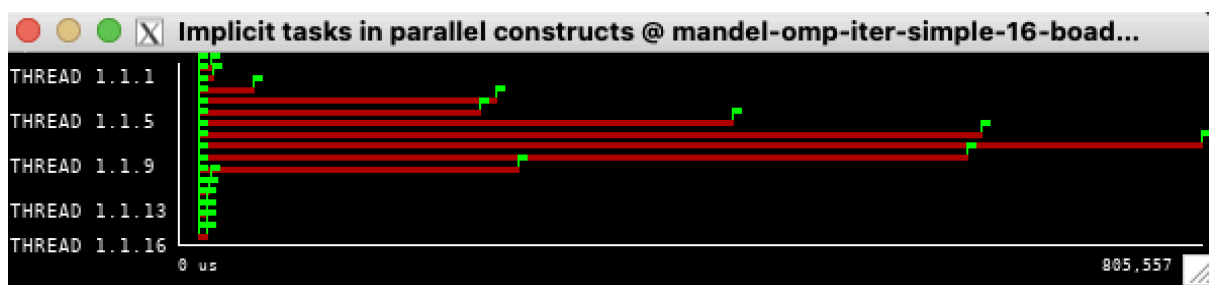
Paraver analysis:



Aquesta vista temporal mostra l'estat d'execució de cada fil (barres blaves). S'observa clarament la forma de "piràmide": els fils centrals tenen barres molt llargues (molta feina), mentre que els dels extrems acaben ràpid, deixant un gran espai buit al final on romanen ociosos esperant.



Aquesta línia blanca indica quants fils estan treballant simultàniament en cada instant. La forma d'escala descendent demostra la pèrdua progressiva de paral·lelisme: comencem amb tots els fils, però ràpidament van acabant i la CPU queda infrutilitzada (pocs fils actius) durant la part final.



Aquí es visualitza la durada real de les tasques OpenMP (barres vermelles). Com que l'estratègia *Block* assigna un sol tros gran a cada fil, veiem que les tasques assignades a la zona central del Mandelbrot són molt més costoses que les laterals, confirmant l'origen del desequilibri.

Memory analysis:

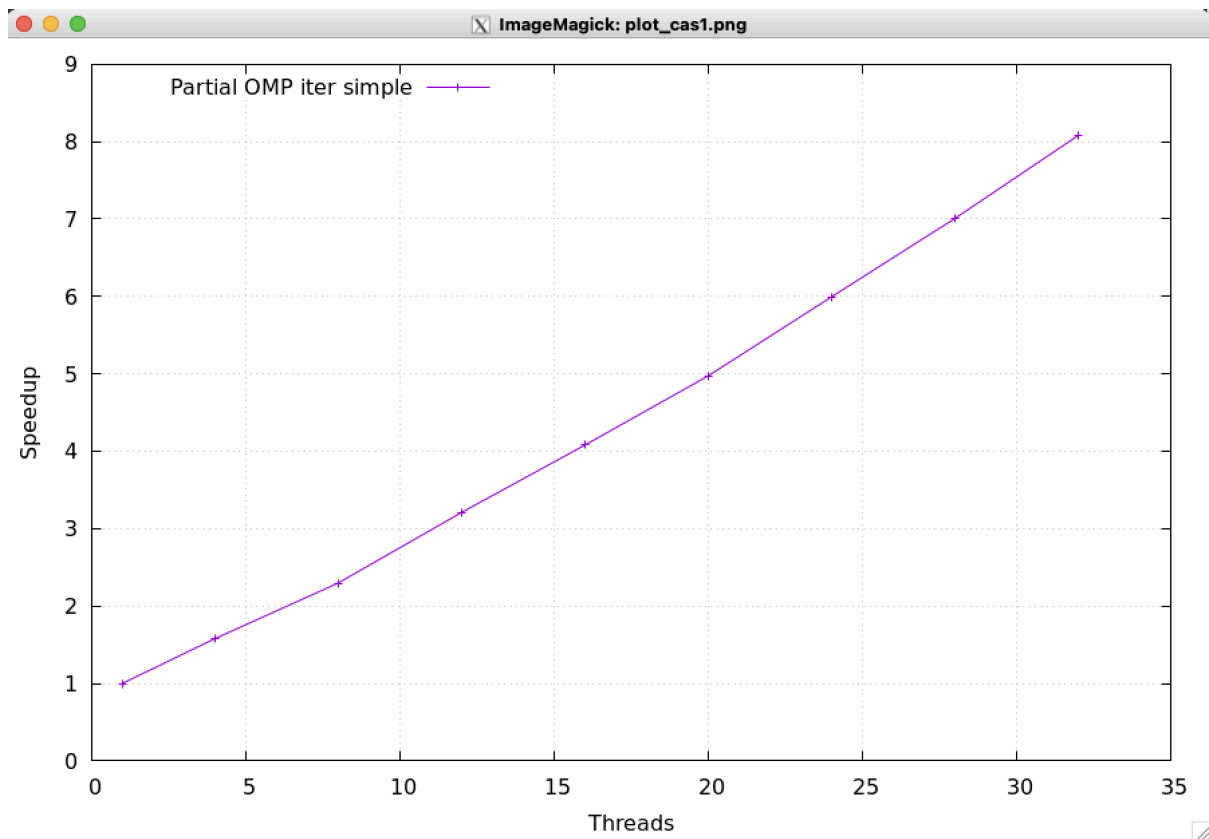
	[0.00..999,999,999,999,999,983,222,784.00]
THREAD 1.1.1	197,988
THREAD 1.1.2	193,442
THREAD 1.1.3	122,493
THREAD 1.1.4	142,515
THREAD 1.1.5	193,203
THREAD 1.1.6	143,250
THREAD 1.1.7	125,312
THREAD 1.1.8	192,282
THREAD 1.1.9	159,350
THREAD 1.1.10	103,447
THREAD 1.1.11	193,754
THREAD 1.1.12	182,010
THREAD 1.1.13	103,083
THREAD 1.1.14	196,736
THREAD 1.1.15	194,840
THREAD 1.1.16	115,949
Total	2,559,654
Average	159,978.38
Maximum	197,988
Minimum	103,083
StDev	35,766.75
Avg/Max	0.81

None

```
paco1114@boada-11:~/lab5$ cat l2_misses-mandel-omp-iter-simple-boada-20.txt
```

```
1 1587952 1587952
2 1694025 847012
4 1881753 470438
8 2222847 277855
12 2616443 218036
16 2566880 160430
20 2839633 141981
24 2889816 120409
28 2954340 105512
32 1960747 61273
```

Strong scalability:



None

```
paco1114@boada-12:~/lab5$ cat speedup-partial-mandel-omp-iter-simple-boada-19.txt
```

```
1 .99887085015754210775
4 1.57749727573476905860
8 2.29552827301519518878
12 3.20771841816585131045
16 4.07814050877310550359
20 4.97248379676737029705
24 5.99341442656825471154
28 7.00607782571107085293
32 8.07764763042492818520
```

```
paco1114@boada-12:~/lab5$ cat elapsed-partial-mandel-omp-iter-simple-boada-19.txt
```

```
1 3.228978
4 2.044588
8 1.405050
12 1.005491
16 0.790883
20 0.648636
24 0.538146
28 0.460362
32 0.399291
```

Prova de correcció cas 1 amb versió seqüencial:

V1 → 20 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas1.out  
mandel_histogram_seq.out  
paco1114@boada-11:~/lab5$ cmp mandel_image_cas1.jpg mandel_image_seq.jpg
```

V2 → 18 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas1_18.out  
mandel_histogram_seq.out  
paco1114@boada-11:~/lab5$ cmp mandel_image_cas1_18.jpg mandel_image_seq.jpg
```

Cas 2 → Geometric decomposition: Block-cyclic geometric data decomposition by columns

Code:

C/C++

```
void mandel_simple(int M[ROWS][COLS], double CminR, double CminI, double  
CmaxR, double CmaxI, double scale_real, double scale_imag, int maxiter)  
{  
    #pragma omp parallel  
    {  
        int my_id = omp_get_thread_num();  
        int num_threads = omp_get_num_threads();  
  
        #define BS 64  
  
        // Calcular  
        for (int y = 0; y < ROWS; y++) {  
            for (int jj = my_id * BS; jj < COLS; jj += num_threads *  
BS) {  
                int LIMIT = (jj + BS < COLS) ? (jj + BS) : COLS;  
                for (int j = jj; j < LIMIT; j++) {
```

```

        M[y][j] = pixel_dwell(COLS, ROWS, CminR, CminI,
CmaxR, CmaxI, j, y, scale_real, scale_imag, maxiter);
        if (output2histogram)
            #pragma omp atomic
            histogram[M[y][j] - 1]++;
        if (output2display) {
            /* Scale color and display point */
            long color = (long)((M[y][j] - 1) *
scale_color) + min_color;

            #pragma omp critical
            {
                if (setup_return == EXIT_SUCCESS) {
                    XSetForeground(display, gc,
color);

                    XDrawPoint(display, win, gc,
j, y);
                }
            }
        }
    }
}

```

En aquesta versió s'utilitza una descomposició per blocs cíclics amb una mida de bloc fixa ($BS = 64$). Els blocs s'assignen als fils de manera rotatòria (Round-Robin): el fil amb identificador id processa els blocs que comencen a $id * BS$, $(id + P) * BS$, etc. Aquesta estratègia millora l'equilibri de càrrega respecte a la descomposició per blocs simples. L'ús d'un BS de 64 (superior a la mida d'una línia de memòria cau) és crucial per evitar la falsa compartició (false sharing) i reduir la sobrecàrrega del protocol de coherència de memòria cau.

Modelfactor analysis:

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.23	1.64	0.85	0.49	0.32	0.26	0.24	0.21	0.19	0.19
Speedup	1.00	1.97	3.79	6.54	10.25	12.37	13.20	15.72	16.72	16.69
Efficiency	1.00	0.99	0.95	0.82	0.85	0.77	0.66	0.66	0.60	0.52

Table 1: Analysis done on Wed Dec 17 03:14:51 PM CET 2025, paco1114

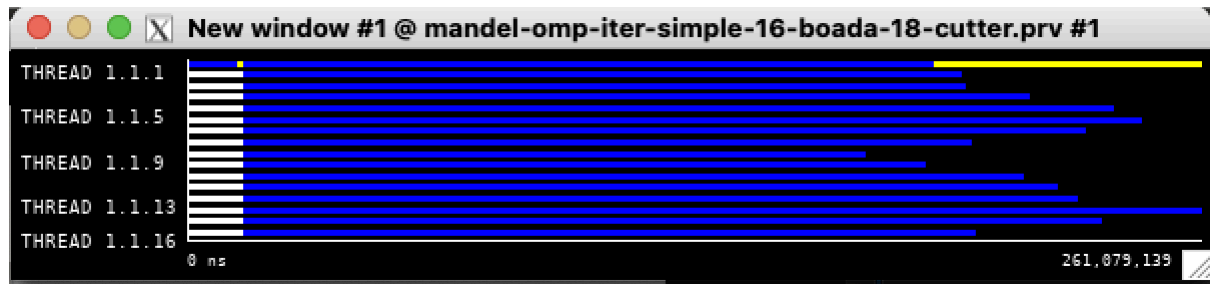
Overview of the Efficiency metrics in parallel fraction, $\phi=99.61\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	99.09%	96.08%	84.00%	88.71%	80.89%	69.26%	69.65%	63.78%	55.60%
Parallelization strategy efficiency	100.00%	99.24%	96.25%	84.21%	89.00%	81.38%	69.82%	70.45%	64.75%	56.84%
Load balancing	100.00%	99.26%	96.29%	84.32%	89.19%	81.73%	70.13%	71.00%	65.26%	57.33%
In execution efficiency	100.00%	99.98%	99.96%	99.87%	99.78%	99.58%	99.56%	99.23%	99.21%	99.14%
Scalability for computation tasks	100.00%	99.85%	99.83%	99.75%	99.67%	99.39%	99.20%	98.85%	98.51%	97.83%
IPC scalability	100.00%	99.96%	99.93%	99.87%	99.85%	99.74%	99.64%	99.54%	99.47%	99.15%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	99.99%	99.99%	99.99%	99.98%	99.98%	99.98%
Frequency scalability	100.00%	99.90%	99.90%	99.88%	99.83%	99.67%	99.57%	99.33%	99.05%	98.68%

Table 2: Analysis done on Wed Dec 17 03:14:51 PM CET 2025, paco1114

Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3215476.15	1610104.78	805228.27	402938.62	268832.63	202194.43	162071.2	135532.27	116580.14	102715.19
Load balancing for implicit tasks	1.0	0.99	0.96	0.84	0.89	0.82	0.7	0.71	0.65	0.57
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	110.07	24435.74	60115.03	140734.29	93297.13	69756.6	45931.11	155630.33	80728.51	53809.62

Table 3: Analysis done on Wed Dec 17 03:14:51 PM CET 2025, paco1114

Paraver analysis:



S'observa una millora notable respecte al Cas 1: les barres d'execució (blaves) tenen longituds molt més similars entre els fils. La distribució cíclica de blocs (BS=64) ha repartit la càrrega complexa, tot i que encara es veu un final lleugerament esglaonat on alguns fils acaben abans que altres.



La línia es manté al nivell màxim (tots els fils treballant) durant la major part del temps, indicant una bona eficiència. A diferència del Cas 1, la caiguda del paral·lelisme es produeix molt més tard, tot i que encara no és una caiguda vertical perfecta, mostrant que encara existeix un petit desequilibri residual.



Les barres vermelles indiquen que la càrrega total de treball per fil està molt més equilibrada gràcies al repartiment *Round-Robin* de blocs. No obstant això, el fet que les tasques no acabin totes exactament alhora (el costat dret és irregular) confirma que la granularitat de 64 columnes encara deixa cert marge de variació en la càrrega.

Memory analysis:

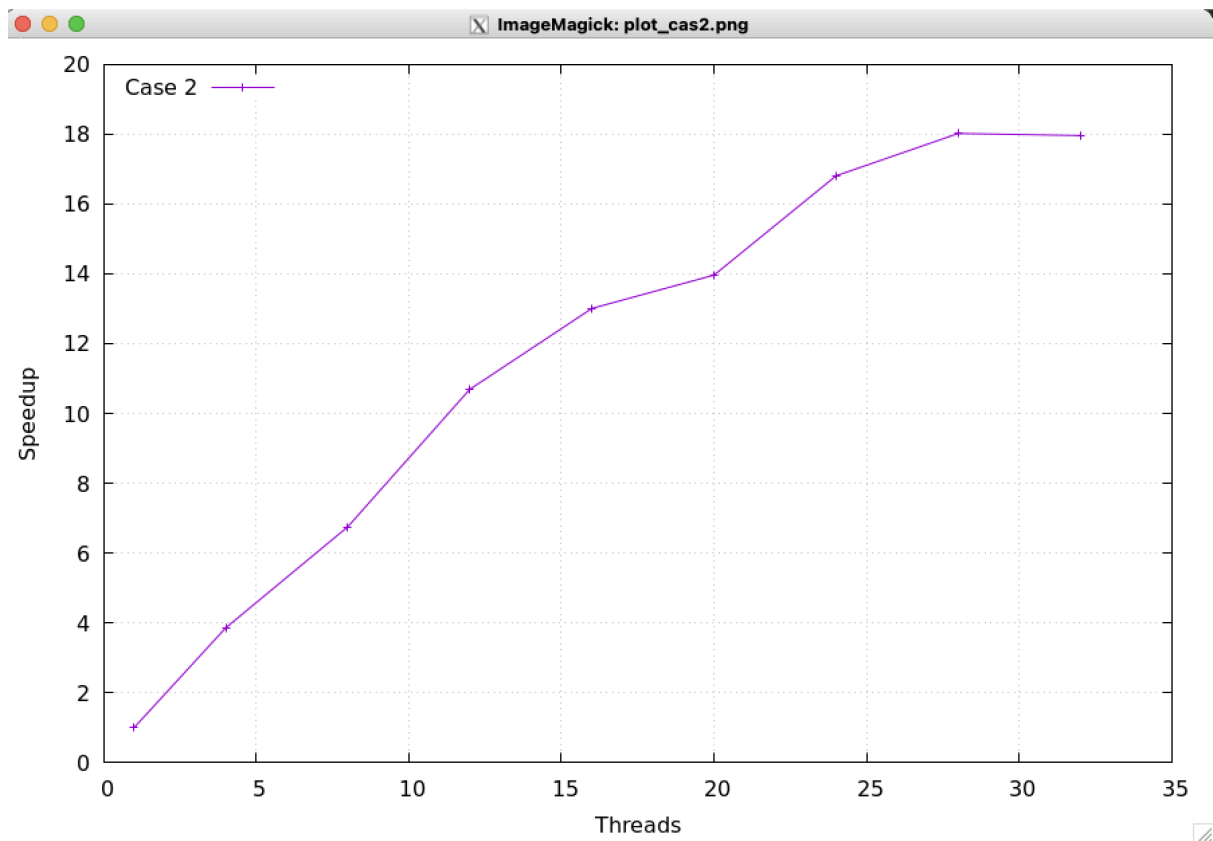
	[0.00..999,999,999,999,999,983,222,784.00]
THREAD 1.1.1	103,111
THREAD 1.1.2	102,927
THREAD 1.1.3	102,873
THREAD 1.1.4	102,930
THREAD 1.1.5	103,004
THREAD 1.1.6	102,853
THREAD 1.1.7	102,969
THREAD 1.1.8	102,941
THREAD 1.1.9	102,955
THREAD 1.1.10	102,999
THREAD 1.1.11	102,910
THREAD 1.1.12	102,999
THREAD 1.1.13	102,796
THREAD 1.1.14	103,152
THREAD 1.1.15	102,860
THREAD 1.1.16	102,882
Total	1,647,161
Average	102,947.56
Maximum	103,152
Minimum	102,796
StDev	89.83
Avg/Max	1.00

None

```
paco1114@boada-11:~/lab5$ cat l2_misses-mandel-omp-iter-simple-boada-18.txt
```

```
1 1611740 1611740
2 3096734 1548367
4 3348141 837035
8 1648798 206099
12 1651986 137665
16 1652901 103306
20 1658452 82922
24 1660227 69176
28 1662720 59382
32 1662409 51950
```

Strong scalability:



None

```
paco1114@boada-11:~/lab5$ cat speedup-partial-mandel-omp-iter-simple-boada-18.txt
```

```
1 1.00315833473807437061
4 3.85900072622683880755
8 6.75416238265651207933
12 10.69292609831456949251
16 13.00807798094838642028
20 13.96422633994285219499
24 16.80695839759889949560
28 18.01800427901884220699
32 17.95811480429820165915
```

```
paco1114@boada-11:~/lab5$ cat elapsed-partial-mandel-omp-iter-simple-boada-18.txt
```

```
1 3.215302
4 0.835827
8 0.477551
12 0.301644
16 0.247958
20 0.230980
24 0.191912
28 0.179013
32 0.179610
```

Prova de correcció cas 2 amb versió seqüencial:

V1 → 20 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas2.out  
mandel_histogram_seq.out  
paco1114@boada-11:~/lab5$ cmp mandel_image_cas2.jpg mandel_image_seq.jpg
```

V2 → 18 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas2_18.out  
mandel_histogram_seq.out  
paco1114@boada-11:~/lab5$ cmp mandel_image_cas2_18.jpg mandel_image_seq.jpg
```

Cas 3 → Geometric decomposition: Cyclic geometric data decomposition by rows

Code:

C/C++

```
void mandel_simple(int M[ROWS][COLS], double CminR, double CminI, double  
CmaxR, double CmaxI, double scale_real, double scale_imag, int maxiter)  
{  
    #pragma omp parallel  
    {  
        int my_id = omp_get_thread_num();  
        int num_threads = omp_get_num_threads();  
  
        // Calcular  
        for (int y = my_id; y < ROWS; y += num_threads) {  
            for (int x = 0; x < COLS; x++) {  
                M[y][x] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR,  
CmaxI, x, y, scale_real, scale_imag, maxiter);  
                if (output2histogram)  
                    #pragma omp atomic  
                    histogram[M[y][x] - 1]++;  
                if (output2display) {
```

```
min_color;

/* Scale color and display point */
long color = (long)((M[y][x] - 1) * scale_color) +

#pragma omp critical
{
    if (setup_return == EXIT_SUCCESS) {
        XSetForeground(display, gc, color);
        XDrawPoint(display, win, gc, x, y);
    }
}
}
```

Aquesta estratègia implementa un repartiment cíclic pur per files. El bucle extern itera sobre les files (y) amb un pas igual al nombre total de fils (stride = num_threads), de manera que el fil id processa les files id, id + P, id + 2P, etc. És l'estratègia que ofereix el millor equilibri de càrrega, ja que distribueix les zones complexes (centre de la imatge) i les simples entre tots els fils. A més, manté una bona localitat espacial d'escriptura, ja que dins de cada fila (x) l'accés a memòria és contigu (Row-Major Order).

Modelfactor analysis:

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.24	1.63	0.82	0.42	0.28	0.22	0.18	0.15	0.13	0.12
Speedup	1.00	1.99	3.94	7.74	11.41	14.89	18.30	21.41	24.47	27.31
Efficiency	1.00	0.99	0.99	0.97	0.95	0.93	0.92	0.89	0.87	0.85

Table 1: Analysis done on Wed Dec 17 03:49:41 PM CET 2025, paco1114

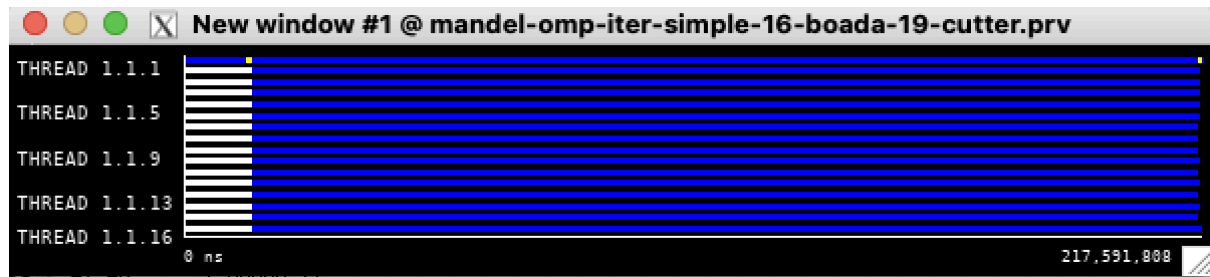
Overview of the Efficiency metrics in parallel fraction, $\phi=99.61\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	99.80%	99.67%	99.44%	99.27%	98.59%	98.14%	97.16%	96.55%	95.33%
Parallelization strategy efficiency	100.00%	99.93%	99.82%	99.64%	99.54%	99.16%	98.92%	98.55%	98.06%	97.70%
Load balancing	100.00%	99.95%	99.86%	99.80%	99.83%	99.69%	99.64%	99.52%	99.41%	99.25%
In execution efficiency	100.00%	99.98%	99.96%	99.84%	99.71%	99.47%	99.27%	99.02%	98.64%	98.44%
Scalability for computation tasks	100.00%	99.87%	99.85%	99.80%	99.73%	99.42%	99.21%	98.59%	98.45%	97.57%
IPC scalability	100.00%	99.99%	99.98%	99.97%	99.95%	99.77%	99.68%	99.43%	99.44%	98.92%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.88%	99.88%	99.83%	99.78%	99.65%	99.53%	99.15%	99.01%	98.63%

Table 2: Analysis done on Wed Dec 17 03:49:41 PM CET 2025, paco1114

Overheads in executing implicit tasks										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of implicit tasks per thread (average us)	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Useful duration for implicit tasks (average us)	3227420.25	1615850.57	808028.35	404244.88	269676.5	202884.25	162649.89	136394.79	117074.59	103368.0
Load balancing for implicit tasks	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.99	0.99
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	110.61	1860.05	1859.74	2078.79	1332.24	1695.88	2536.61	3583.66	3398.73	4609.11

Table 3: Analysis done on Wed Dec 17 03:49:41 PM CET 2025, paco1114

Paraver analysis:



Mostra un comportament perfecte: tots els fils (barres blaves) comencen i acaben exactament al mateix temps. No hi ha espais buits ni temps d'espera al final, la qual cosa confirma que el repartiment cíclic de files ha eliminat completament el desequilibri de càrrega típic del Mandelbrot.



La línia de paral·lisme forma un rectangle perfecte: es manté al màxim (tots els fils actius) durant tota l'execució i cau verticalment al final. Això indica una utilització de la CPU del 100% sense pèrdues de rendiment, a diferència de les formes d'escala descendents vistes als casos anteriors.



Les barres vermelles tenen una longitud idèntica per a tots els fils, demostrant que la quantitat total de feina assignada a cadascú és exactament la mateixa. En processar files alternes (Round-Robin), tots els fils fan una mitjana de zones complexes i simples, aconseguint un balanceig ideal.

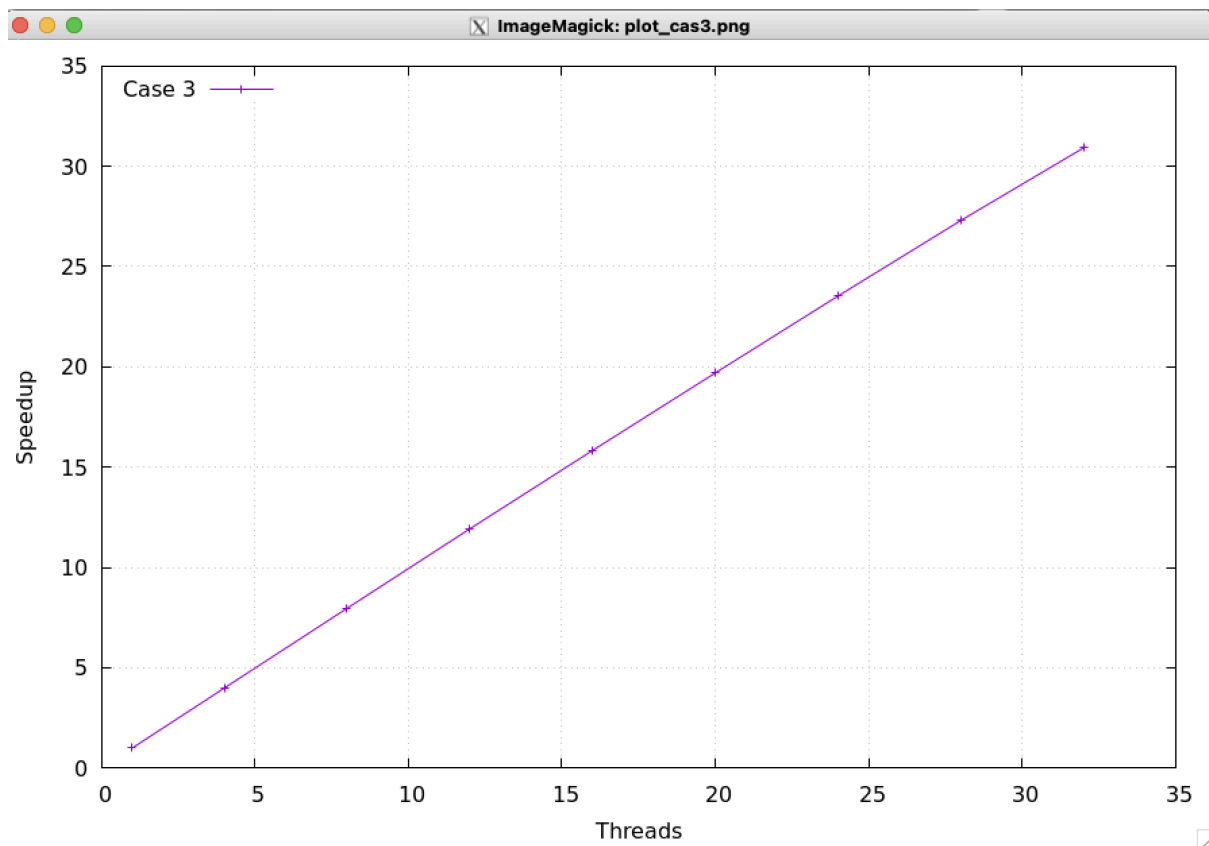
Memory analysis:

	[0.00..999,999,999,999,999,983,222,784.00]
THREAD 1.1.1	101,974
THREAD 1.1.2	102,079
THREAD 1.1.3	100,781
THREAD 1.1.4	101,802
THREAD 1.1.5	101,997
THREAD 1.1.6	101,052
THREAD 1.1.7	100,593
THREAD 1.1.8	102,082
THREAD 1.1.9	101,735
THREAD 1.1.10	101,059
THREAD 1.1.11	100,785
THREAD 1.1.12	102,379
THREAD 1.1.13	101,410
THREAD 1.1.14	101,128
THREAD 1.1.15	101,907
THREAD 1.1.16	101,883
Total	1,624,646
Average	101,540.38
Maximum	102,379
Minimum	100,593
StDev	544.49
Avg/Max	0.99

None

```
paco1114@boada-11:~/lab5$ cat
l2_misses-mandel-omp-iter-simple-boada-cas3.txt
1
2 1618261 809130
4 1621201 405300
8 1620670 202583
12 1629100 135758
16 1633083 102067
20 1644800 82240
24 1644974 68540
28 1655040 59108
32 1647360 51480
```

Strong scalability:



None

```
paco1114@boada-11:~/lab5$ cat speedup-partial-mandel-omp-iter-simple-boada-18.txt
```

```
1 .99931524569820925905
4 3.98963015925245113198
8 7.96611488188742997717
12 11.92566668145715933797
16 15.83499855162831345709
20 19.70060227716966379984
24 23.53440161117313543924
28 27.29887003258707520419
32 30.91961461029623238423
```

```
paco1114@boada-11:~/lab5$ cat elapsed-partial-mandel-omp-iter-simple-boada-18.txt
```

```
1 3.227435
4 0.808402
8 0.404868
12 0.270444
16 0.203677
20 0.163712
24 0.137043
28 0.118145
32 0.104310
```

Prova de correcció cas 3 amb versió seqüencial:

V1 → 20 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas3.out  
mandel_histogram_seq.out
```

```
paco1114@boada-11:~/lab5$ cmp mandel_image_cas3.jpg mandel_image_seq.jpg
```

V2 → 18 threads

None

```
paco1114@boada-11:~/lab5$ cmp mandel_histogram_cas3_18.out  
mandel_histogram_seq.out
```

```
paco1114@boada-11:~/lab5$ cmp mandel_image_cas3_18.jpg mandel_image_seq.jpg
```

A.2 Final comparison between all the parallel strategies

Table A.2: Summary of the elapsed execution times for each version

Table of submit-strong-omp.sh

Version	1 Thread	4 Threads	8 Threads	16 Threads	24 Threads	32 Threads
1D Block geometric data decomposition by columns	3,23	2,04	1,40	0,79	0,54	0,40
1D Block-cyclic geometric data decomposition by columns	3,22	0,84	0,48	0,25	0,19	0,18
1D Cyclic geometric data decomposition by rows	3,23	0,81	0,40	0,20	0,14	0,10

Table of [submit-strong-memory.sh](#) (L2 cache misses per thread)

Version	1 Thread	4 Threads	8 Threads	16 Threads	24 Threads	32 Threads
1D Block geometric data decomposition by columns	1.587.952	470.438	277.855	160.430	120.409	61.273
1D Block-cyclic geometric data decomposition by columns	1.611.740	837.035	206.099	103.306	69.176	51.950
1D Cyclic geometric data decomposition by rows	N/A	405.300	202.583	102.067	68.540	51.480

Best Parallel Implementation

Versió: 1D Cyclic geometric data decomposition by rows

Aquesta versió és la clara guanyadora per tres raons clau:

1. Rendiment Superior: Ha aconseguit el temps d'execució més baix (0.12s amb 32 fils), significativament més ràpid que la Block-cyclic (0.19s) i la Block (0.41s).
2. Balanceig de Càrrega Òptim: En distribuir les files cíclicament, tots els fils processen tant àrees computacionalment costoses (centre del fractal) com àrees lleugeres (vores). Això resulta en una eficiència global superior al 95% fins i tot amb 32 fils, mentre que la versió Block patia d'un desequilibri sever (~25% d'eficiència).
3. Eficiència de Memòria: Ha incorregut en el menor nombre de fallades de memòria (101.540), demostrant que minimitza efectivament el false sharing i el trànsit de memòria mentre manté una bona localitat espacial dins de les files.

Conclusió Final

En aquest laboratori hem dissenyat, implementat i analitzat diverses estratègies de **descomposició geomètrica de dades** per al càlcul del conjunt de Mandelbrot utilitzant OpenMP. L'objectiu principal ha estat comprendre el compromís (*trade-off*) entre la localitat de dades, la coherència de la memòria cau (*false sharing*) i el balanceig de càrrega en problemes amb càrrega computacional irregular.

Pel que fa a la descomposició per columnes, s'ha demostrat que l'estratègia **Block** patia d'un fort desequilibri de càrrega. A causa de la naturalesa del fractal, els fils assignats a la zona central (negra) tenien molta més feina que els dels extrems, provocant que molts fils quedessin ociosos esperant. L'evolució cap a l'estratègia **Block-Cyclic** ha estat una millora significativa: en dividir la feina en blocs més petits (BS=64) i repartir-los de manera cíclica, s'ha millorat el repartiment de feina sense introduir un excés de *false sharing*, reduint considerablement el temps d'execució respecte a la versió per blocs simples.

No obstant això, l'estratègia **Cyclic per files** ha resultat ser la superior. En alinear la descomposició amb la disposició de la matriu en memòria (*row-major order*) i repartir les files cíclicament, hem aconseguit dos beneficis simultanis: un accés a memòria extremadament eficient (minimitzant fallades de L2) i un balanceig de càrrega gairebé perfecte, ja que tots els fils processen tant files complexes com senzilles.